

Tipos de Datos en Python

Fundamentos, Árboles de Memoria y Aplicaciones Industriales

Enteros (int)

42, -100, 0

Flotantes (float)

3.14159, -0.01

Texto (str)

"PLC_Sensor_01"

Listas (list)

[25.4, 25.8, 26.1]

Diccionarios

{"id": 101, "val": 4.2}

Curso: Curso Profesional de Python

Unidad: Módulo 1 - Fundamentos de Programación y Gestión de Memoria

Nivel: Desde Principiante Absoluto hasta Nivel Práctico Empresarial

Enfoque: Desarrollo de Software, Automatización Industrial e Ingenierías

Autoría: Departamento de Ingeniería y Pedagogía de Software

1. Objetivo del Capítulo

El objetivo principal de este capítulo es brindar al estudiante una comprensión rigurosa, profunda y práctica de la naturaleza y manipulación de datos en el lenguaje de programación Python. Al finalizar este capítulo, el estudiante comprenderá exactamente cómo la computadora almacena, procesa y transforma información en la memoria RAM, eliminando la incertidumbre conceptual al escribir código.

Al finalizar este capítulo serás capaz de:

- Diferenciar y manipular correctamente los tipos de datos primarios (``int``, ``float``, ``str``, ``bool``, ``bytes``).
- Seleccionar la estructura de datos ideal (``list``, ``tuple``, ``dict``, ``set``) según la necesidad de rendimiento e inmutabilidad.
- Explicar el modelo de memoria de Python: referencias, punteros de objeto y diferencia entre mutabilidad e inmutabilidad.
- Realizar conversiones de tipos (Type Casting) explícitas e implícitas evitando pérdidas de precisión.
- Diagnosticar y corregir errores comunes en producción como ``TypeError``, ``ValueError`` e ``IndexError``.
- Escribir programas robustos aplicando anotaciones de tipo (``Type Hints``) bajo estándares profesionales PEP 8.

2. Introducción al Tema

¿Qué son los Tipos de Datos en Python?

Un **Tipo de Dato** es una clasificación que le indica al interprete de Python (y a la CPU) qué clase de valor está procesando, qué cantidad de espacio de memoria requiere y qué operaciones matemáticas o lógicas están permitidas sobre él. En la vida real, una caja con la etiqueta "Fragil: Cristal" recibe un trato totalmente distinto al de un contenedor de acero. De manera idéntica, un texto no puede multiplicarse algebraicamente como se haría con un número de temperatura.

¿Por qué existe y qué problema resuelve?

Los procesadores solo entienden estados binarios (``0`` y ``1``). Sin los tipos de datos, la secuencia de ceros y unos ``01000001`` sería ambigua: podría representar el número entero ``65``, la letra ``A``, o el estado de activación de un relé industrial. Los tipos de datos introducen un *esquema de semántica y reglas* para interpretar los bytes de la memoria en conceptos del mundo real.

Analogía de la vida real

Imagina un centro logístico de envíos. Tienes **contenedores de líquidos, cajas de texto/documentos y palets de productos en inventario**. No puedes meter un archivo en un tubo de gas. Los tipos de datos le dicen a Python qué contenedor usar y qué herramientas (métodos) utilizar sobre cada objeto.

¿Qué ocurriría si este concepto no existiera?

Sin tipos de datos, la programación sería caótica e insegura. Ocurrirían colisiones catastróficas: el sistema intentaría sumar un nombre de cliente a un saldo bancario, corruptos los archivos de la memoria o colapsando los procesadores industriales al procesar datos de sensores incoherentes.

3. Explicación Completa: Tipos Primarios

A. Números Enteros (`int`)

¿Qué es? Representa números enteros sin parte decimal, tanto positivos como negativos, y el cero.

¿Por qué y cuándo se utiliza? Se utiliza para contar elementos discretos: cantidad de paquetes en banda transportadora, número de usuarios, ciclos de motor o IDs de base de datos.

¿Cómo funciona internamente? En Python, los enteros tienen *precisión arbitraria*. A diferencia de C o Java (limitados a 32 o 64 bits), Python asigna memoria dinámicamente para almacenar enteros tan grandes como la RAM lo permita.

```
# Ejemplo de Enteros y medición de memoria piezas_producidas = 1450 unidades_defectuosas
```

B. Números de Punto Flotante (`float`)

¿Qué es? Representa números reales con parte fraccionaria o decimal.

¿Por qué y cuándo se utiliza? Para magnitudes continuas: temperaturas, voltajes de sensores, mediciones de presión, presupuestos monetarios de alta precisión o coordenadas GPS.

¿Cómo funciona internamente? Implementa el estándar **IEEE 754** de doble precisión (64 bits). Esto significa que tiene un límite de 53 bits de mantisa (~15 a 17 dígitos decimales de precisión).

Atención: Imprecisión en Flotantes

Operaciones como $0.1 + 0.2$ devuelven `0.30000000000000004` debido a la representación binaria fraccionaria. Para finanzas críticas o metrología de alta precisión se recomienda el módulo estándar `decimal`.

C. Cadenas de Texto (`str`)

¿Qué es? Secuencia inmutable de caracteres Unicode.

Sintaxis: Se delimita mediante comillas simples (`'...'`), dobles (`"..."`) o triples (`"""..."""`).

```
# Ejemplo de manipulación de texto profesional codigo_sensor = " SEN-TEMP-089 " codigo_li
```

4. Tipos Lógicos y de Bytes + Modelo de Memoria

D. Booleanos (`bool`)

¿Qué es? Tipo de dato que representa únicamente uno de dos valores lógicos: True (Verdadero) o False (Falso).

¿Cómo funciona internamente? En Python, la clase `bool` es una subclase directa de `int`. True equivale numéricamente a 1 y False a 0.

```
# Evaluación booleana en automatización temperatura_actual = 88.5 umbral_critico = 80.0 a
```

E. Bytes y Bytearrays (`bytes`)

¿Qué es? Secuencia binaria cruda (valores enteros entre 0 y 255). Es crucial para la comunicación directa con hardware, sockets de red y archivos binarios.

```
# Trama binaria simulando protocolo MODBUS RS485 trama_binaria = b"\x01\x03\x00\x00\x00\x
```

5. Glosario Técnico: Modelo de Memoria en Python

Para entender Python a nivel profesional, es fundamental dominar su arquitectura interna de manejo de datos:

Tipado Dinámico

No se declara explícitamente el tipo de una variable. Python detecta automáticamente el tipo del objeto asignado en tiempo de ejecución.

Tipado Fuerte

Python no realiza conversiones explícitas incompatibles en operaciones. "5" + 5 produce un `TypeError` explícito.

Punteros y Referencias de Memoria (`id()`)

Las variables en Python no contienen los datos directamente; contienen **referencias de memoria (punteros)** que apuntan a objetos en el *Heap*. La función integrada `id(objeto)` devuelve la dirección de memoria única del objeto.

```
# Demostración de referencias de memoria x = 1000 y = x # 'y' apunta a la misma direc
```

6. Mutabilidad vs Inmutabilidad

Este es el concepto crucial que separa a los principiantes de los desarrolladores experimentados.

- **Objeto Inmutable:** Un objeto cuyo estado o contenido no puede ser modificado después de su creación (ej. `int`, `float`, `str`, `tuple`, `bytes`). Modificarlo crea un nuevo objeto en otra dirección de memoria.
- **Objeto Mutable:** Un objeto cuya estructura interna puede alterarse in situ (in-place) sin cambiar su ID de memoria (ej. `list`, `dict`, `set`).

INMUTABLE (str)

```
texto = "Hola"
texto += " Mundo"
→ Crea NUEVO objeto en memoria
```

MUTABLE (list)

```
lista = [1, 2]
lista.append(3)
→ Modifica la MISMA memoria
```

7. Colecciones y Estructuras de Datos

A. Listas (`list`)

¿Qué es? Secuencia ordenada y **MUTABLE** de elementos heterogéneos.

```
# Operaciones sobre listas en un inventario de almacén stock = ["Motor AC", "Válvula 1/2"]
```

B. Tuplas (`tuple`)

¿Qué es? Secuencia ordenada e **INMUTABLE**. Garantiza la integridad de datos que no deben cambiar a lo largo del programa.

```
# Configuración inalterable de conexión a servidor industrial config_server = ("192.168.1
```

8. Diccionarios y Conjuntos

C. Diccionarios (`dict`)

¿Qué es? Colección de pares clave: valor, **MUTABLE** y altamente optimizada mediante tablas Hash (búsqueda en tiempo constante $O(1)$).

Regla de Claves: Las claves deben ser de un tipo *immutable* (hashable) como str, int o tuple.

```
# Registro de telemetría de máquina industrial sensor_data = { "id_sensor": "TEMP_ZONE_01"
```

D. Conjuntos (`set`)

¿Qué es? Colección **MUTABLE**, desordenada de elementos **únicos** (sin duplicados). Excelente para operaciones matemáticas de conjuntos (unión, intersección, diferencia).

```
# Filtrado automático de RFID de empleados en planta fichajes_mañana = {"EMP_01", "EMP_02"
```

9. Tabla Comparativa de Estructuras

Tipo	Sintaxis	Ordenado	Mutable	Duplicados	Caso de Uso Principal
List	[1, 2, 3]	Sí	SÍ	Sí	Listas dinámicas de elementos secuenciales.
Tuple	(1, 2, 3)	Sí	NO	Sí	Coordenadas, registros fijos, constantes.
Dict	{"k": "v"}	Sí (3.7+)	SÍ	Claves No / Val Sí	Entidades con atributos (Mapeos JSON).
Set	{1, 2, 3}	No	SÍ	No	Eliminar duplicados, membresía veloz.

10. Ejemplos Progresivos de Código

Nivel 1: Conversión Explícita (Básico)

```
# Entrada de datos desde terminal o interfaz gráfica siempre devuelve str entrada_usuario
```

Análisis Línea por Línea:

- Línea 2: Asigna la cadena "150" a la variable `entrada_usuario`.
- Línea 3: Se invoca `int()` para transformar el texto en entero numérico.
- Línea 4: Se efectúa la operación matemática directa sobre el valor entero.

Nivel 2: Integración con Colecciones (Intermedio)

```
# Procesamiento de mediciones de un sensor de humedad lecturas = [45.2, 48.1, 51.0, 44.9]
```

Nivel 3 & 4: Sistema Empresarial / Industrial (Profesional)

```
from typing import Dict, List, Union # Procesamiento de lote de fabricación de componente
```

11. Errores Comunes y Cómo Evitarlos

1. TypeError: Concatenación Incompatible

El Error: Intentar sumar un texto y un entero directamente: `"Edad: " + 25`.

Causa: Python es de tipado fuerte y no convierte números a cadenas automáticamente en suma.

Solución Profesional: Usar Formato de Cadenas F-Strings: `f"Edad: {25}"` o conversión explícita `"Edad: " + str(25)`.

2. Mutation Bug: Modificación Involuntaria por Referencia

El Error: `lista_b = lista_a`. Modificar `lista_b` altera sorpresivamente `lista_a`.

Causa: Ambas variables apuntan exactamete a la misma dirección de memoria RAM.

Solución: Crear una copia explícita (Shallow Copy): `lista_b = lista_a.copy()` o `lista_b = list(lista_a)`.

12. Buenas Prácticas de Desarrollo (PEP 8 & Clean Code)

- **Uso de Type Hints:** Declara siempre los tipos esperados en funciones para habilitar autocompletado y validación estática con mypy:
`def calcular_eficiencia(potencia_kw: float, horas: int) -> float:`
- **Nombres Descriptivos:** Usa prefijos lógicos para booleanos: `es_activo`, `tiene_permiso`, `motor_encendido`.
- **Inmutabilidad Defensiva:** Pasa tuplas o conjuntos congelados (`frozenset`) a funciones para prevenir modificaciones accidentales de colecciones globales.
- **Constantes Explícitas:** Define valores fijos en mayúsculas: `VOLTAJE_MAXIMO = 230.0`.

13. Ejercicios Prácticos

Resuelve los siguientes ejercicios individualmente antes de consultar las respuestas.

Ejercicio 1: Comprensión Básica (Type Casting)

Enunciado: Dado el siguiente dato recibido de una balanza digital: `peso_str = "12.450"`. Conviértelo a flotante, calcula el peso de un lote de 15 cajas idénticas y muestra el resultado como un número entero redondeado.

Pista: Combina `float()`, multiplicación, `round()` e `int()`.

Ejercicio 2: Aplicación (Limpieza de Lista)

Enunciado: Un scanner RFID ha registrado los siguientes accesos con duplicados: `["ID01", "ID02", "ID01", "ID03", "ID02", "ID04"]`. Escribe un código para obtener la cantidad de usuarios *únicos* que ingresaron al edificio.

Ejercicio 3: Integración (Estructuración de Datos)

Enunciado: Diseña un diccionario que represente un robot industrial. Debe contener: su modelo (string), estado activo (booleano), coordenadas XYZ (tupla) y lista de componentes instalados (lista). Modifica el estado del robot a inactivo y añade un componente más a la lista.

Ejercicio 4: Caso Práctico (Filtro de Telemetría)

Enunciado: Dada una lista de lecturas de temperatura: `[18.5, 22.0, 85.4, 19.1, 90.2, 21.3]`. Extrae en una nueva lista únicamente las temperaturas consideradas anomalías (superiores a 50.0°C).

14. Mini Proyecto del Capítulo

Proyecto: Sistema de Monitoreo de Sensores e Inventario Industrial

Contexto: En una planta embotelladora automatizada, se requiere procesar los datos de telemetría de la línea de llenado para auditar el rendimiento y descartar lotes defectuosos.

Requisitos Funcionales:

1. Almacenar la información de los sensores en una estructura de datos adecuada.
2. Filtrar lecturas fuera de rango operacional.
3. Consolidar un reporte de estado en un diccionario.

Estructura del Proyecto Paso a Paso:

```
# PASO 1: Definición de la estructura de datos base base_datos_planta = { "linea_01": { "
```

15. Comprueba tus Conocimientos (Autoevaluación)

1. ¿Cuál es la diferencia fundamental en memoria entre una Tupla y una Lista?
 - a) La tupla solo acepta números.
 - b) La tupla es inmutable y la lista es mutable.
 - c) La lista no permite elementos duplicados.
2. ¿Qué resultado produce la instrucción `type(3.0 + 2)` en Python?
 - a) `<class 'int'>`
 - b) `<class 'float'>`
 - c) `<class 'str'>`
3. ¿Qué sucede si intentas usar una lista como clave de un diccionario?
 - a) Se crea el diccionario correctamente.
 - b) La lista se convierte automáticamente a tupla.
 - c) Ocurre un error `TypeError: unhashable type: 'list'`.
4. ¿Qué estructura garantiza la unicidad de sus elementos eliminando duplicados?
 - a) `list`
 - b) `set`
 - c) `dict`
5. ¿Qué devuelve la expresión `bool("")`?
 - a) `True`
 - b) `False`
 - c) `None`

Respuestas y Explicaciones Detalladas

1. **Resp: b)** Las tuplas son inmutables (no pueden modificarse tras su creación), lo que otorga mayor velocidad de ejecución y seguridad en los datos.
2. **Resp: b)** Coerción implícita: Al operar un `int` con un `float`, Python promociona automáticamente el resultado a `float` para evitar pérdida de decimales.
3. **Resp: c)** Las claves de un diccionario deben ser objetos inmutables ("hashables"). Al ser la lista mutable, su valor Hash puede cambiar, rompiendo la tabla Hash.
4. **Resp: b)** Los conjuntos (`set`) implementan reglas matemáticas de conjuntos y rechazan cualquier valor duplicado.
5. **Resp: b)** En evaluación booleana, las cadenas vacías `""` son consideradas valores "Falsy" (falsos).

16. Resumen del Capítulo e Ideas Clave

Concepto	Tipo Python	Ejemplo	Regla de Oro
Entero	int	100, -5	Precisión infinita. Usado para conteos discretos.
Flotante	float	3.1416	Usa IEEE 754. Cuidado con redondeos financieros.
Cadena	str	"PLC_01"	Inmutable. Admite Unicode completo.
Lista	list	[1, "a", True]	Mutable, ordenada. Ideal para colecciones dinámicas.
Tupla	tuple	(10, 20)	Inmutable, ordenada. Ideal para registros protegidos.
Diccionario	dict	{"id": 1}	Pares clave-valor. Búsqueda instantánea $O(1)$.

Ideas Clave que Debes Recordar:

- En Python **todo es un objeto** alojado en el Heap de la memoria RAM.
- Las variables son **etiquetas de referencia (punteros)** que apuntan a objetos, no contenedores físicos.
- Distingue siempre entre **objetos mutables** (listas, dicts, conjuntos) e **inmutables** (enteros, floats, cadenas, tuplas).
- Usa Type Hints para documentar profesionalmente tu código y prevenir errores en grandes proyectos.

17. Mapa de Aprendizaje Progresivo



¡Felicidades por completar el Capítulo 1! Estás listo para avanzar hacia el **Capítulo 2: Estructuras de Control de Flujo y Lógica Condicional**.